

신경망 기초, 역전파, 학습 기법

퍼셉트론의 한계와 역전파 · 그래디언트 소실 · 폭발과 활성화함수 · 초기화 · 정규화와 학습률 스케줄링

Machine Learning 1 · 9주차

한국과학영재학교 (KSA)

Chapter 9

이번 주차 개요

이 챕터를 마치면

- XOR이 직선 하나로 안 풀리는 이유를 설명하고, 은닉층이 어떻게 푸는지 보여준다
- 역전파를 연쇄법칙으로부터 유도하고, 구현을 수치 미분으로 검증한다
- 그래디언트 소실·폭발을 진단하고, 활성화 함수와 초기화를 고른다
- 정규화와 학습률 스케줄링을 과적합·불안정 학습 상황에 맞게 적용한다
- **9.1 퍼셉트론의 한계, 순전파, 역전파** — XOR의 벽, 오차 δ 의 역전파, 2-2-1 손계산, numpy로 XOR 풀기
- **9.2 그래디언트 소실·폭발, 활성화 함수, 초기화** — 연쇄법칙의 곱, ReLU 계열, Xavier/He 초기화, “세 라인 방어선”
- **9.3 정규화 기법과 학습률 스케줄링** — dropout, BatchNorm, 가중치 감쇠, StepLR/코사인/warmup

세 절은 한 사슬: 그래디언트는 “연쇄법칙의 곱”(9.1) → 깊어지면 0으로 사라지거나 불어나는(9.2) → 학습을 안정적으로 끝내는 도구(9.3). Block B(CNN → RNN → 트랜스포머 → LLM)의 서막이다.

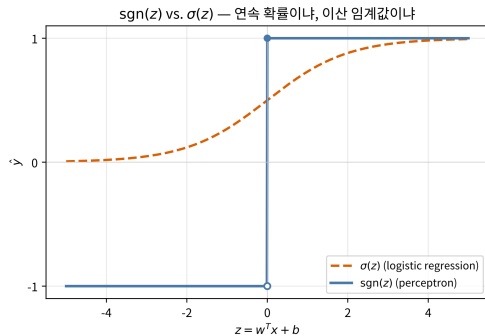
9.1 퍼셉트론의 한계, 순전파, 역전파

퍼셉트론: “AI의 시작”이자 “AI의 실망”

- 1958, 로젠블라트(코넬): 퍼셉트론 발표 — “컴퓨터가 배운다”, “우주 탐사 1세대 로봇”
- Chapter 2의 로지스틱회귀와 거의 동일, 시그모이드 대신 **스텝함수**

$$\hat{y} = \text{sgn}(w^T x + b)$$

- 실제로 (당시 기준) 조종사 얼굴 인식 실험까지 수행



sgn(파랑, $z = 0$ 에서 도약) vs 시그모이드(주황, 매끄러운 전이)

퍼셉트론 학습 규칙 (경사하강법의 선조): 틀린 예제에만 오차 방향으로 갱신, $w \leftarrow w + yx$ — 선형분리 가능 데이터에서 **반드시 수렴**(수렴 정리)

1969, Perceptrons: XOR의 벽과 AI의 겨울

- **민스키·페퍼트(MIT, 1969)**: 단층 퍼셉트론은 XOR을 아무리 학습시켜도 절대 못 맞춘다 — 수학적으로 증명
- “학습률 부족”, “데이터 부족”이 아니라 **모델 구조 자체가 불가능**

영향: 투자·논문 급감 → 1차 “AI 겨울”(1970s 후반~1980s 중반)의 직접 원인. 대중적으로는 “신경망은 쓸모없다”로 과도하게 일반화됐다.

- 그 긴 동면기를 깨운 것: 1986년 **럼멜하트-힌튼-윌리엄스**의 **역전파** 대중화 논문
- 동면기를 깨운 것이 바로 “XOR의 벽을 넘는 단 하나의 구조” — 다음 프레임의 **은닉층**

퍼셉트론에서 다층 신경망(MLP)으로

- 해법: 직선 하나로 안 되면, 직선을 여러 개 겹쳐 새 공간을 만든 뒤 그 공간에서 다시 나누기
- 로지스틱회귀 = 은닉층 없는 가장 단순한 “신경망” $a = \sigma(w^T x)$
- **MLP**: 그 사이에 **은닉층**을 하나 이상 끼워 넣음
- 은닉층 하나만 추가해도 XOR을 완벽히 풀 수 있다

x_1	x_2	XOR
0	0	0
0	1	1
1	0	1
1	1	0

라벨 1인 점 (0, 1), (1, 0)과 라벨 0인 점 (0, 0), (1, 1)이 대각선으로 교차 배치 — 어떤 직선을 그어도 서로 다른 라벨이 한쪽에 섞여 들어간다.

XOR의 부등식 모순 — 직접 세워보자

직선분리 가능하다는 말 = 어떤 (w_1, w_2, b) 가 다음을 **모두** 만족:

$$w_2 + b > 0 \quad (0, 1) \quad w_1 + b > 0 \quad (1, 0) \quad b \leq 0 \quad (0, 0) \quad w_1 + w_2 + b \leq 0 \quad (1, 1)$$

- ① 첫 두 식을 더하면 $w_1 + w_2 + 2b > 0$
- ② 셋째 식($b \leq 0$)과 결합 $\Rightarrow w_1 + w_2 + b > -b \geq 0$
- ③ 넷째 식은 $w_1 + w_2 + b \leq 0$ 이라 주장 — **모순**

결론

부등식 네 개는 어떤 (w_1, w_2, b) 로도 동시에 성립 불가. 퍼셉트론에 필요한 것은 **비선형 결정경계**.

은닉층이 실제로 하는 것: “새 공간” 만들기

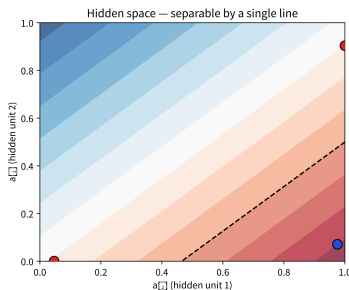
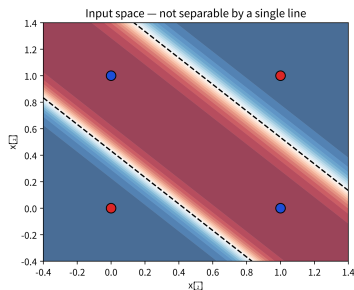
은닉 유닛 2개를 $f_1 = \sigma(x_1 + x_2 - 1)$, $f_2 = \sigma(-x_1 - x_2 + 1)$ 로 정하면:

(x_1, x_2)	라벨	f_1	f_2
(0, 0)	0	≈ 0.27	≈ 0.73
(1, 1)	0	≈ 0.73	≈ 0.27
(0, 1)	1	0.5	0.5
(1, 0)	1	0.5	0.5

- 새 공간 (f_1, f_2) 에서: **라벨 1의 두 점은 한 점으로 모이고**, 라벨 0의 두 점은 좌표가 다름
- 이제 **직선 하나로 깔끔히 분리**

중요한 것은 “이런 가중치가 존재한다”는 것 — 그리고 역전파가 실제로 그 가중치를 찾는다(실습: 은닉 4개로 학습).

“새로운 공간”이 그림으로: 결정경계



- 입력 공간 (x_1, x_2) : 네 점 사이를 비선형 곡선이 지나 — 직선 하나로 못 나눔
- 은닉 공간 (a_1, a_2) : 같은 네 점이 직선 하나로 깔끔히 분리

은닉층의 역할

“새 공간을 만들어, 거기서 직선으로 나누는 것.”

(실제로 학습된 망, 은닉 유닛 2개 · 20,000 에폭)

순전파 (Forward Propagation)

2층 신경망(입력 $\rightarrow$ 은닉 $\rightarrow$ 출력, W_1, b_1 은닉층 / W_2, b_2 출력층):

$$z_1 = W_1x + b_1, \quad a_1 = \sigma(z_1), \quad z_2 = W_2^\top a_1 + b_2, \quad a_2 = \sigma(z_2)$$

- z : **활성화 전**(pre-activation), a : **활성화 후** — 최종 예측값은 a_2

주의 — 컨벤션

$W_1 \in \mathbb{R}^{m \times d}$ (행=은닉 유닛, 열=입력)이면 $z_1 = W_1x + b_1$ 이 자연스럽고,

PyTorch/TensorFlow의 Linear와 일치.

아래 손계산은 **전치 배치**(행=입력, 열=은닉)로 $z_1 = x^\top W_1 + b_1$ 를 쓴다 — “같은 값이 전치된 모습으로 쓰여 있는” 것이 수치 미분 검증·9.2절의 가장 흔한 혼동의 근원.

역전파: 연쇄법칙을 거꾸로

- 새 문제: 은닉층 가중치를 어떻게 학습할 것인가 — 은닉층에는 정답을 직접 비교할 대상이 없다
- **역전파**(rumelhart-hinton-williams, 1986): 출력층의 오차를 **연쇄법칙으로 거꾸로** 흘려보내, 각 가중치가 최종 오차에 얼마나 책임이 있는지 정확히 계산

$z_1 \rightarrow a_1 \rightarrow z_2 \rightarrow a_2 \rightarrow L$ (이 긴 사슬을 출력에서 입력 방향으로 거꾸로 따라가며 미분을 곱)

- ① **출력층 오차:** $\delta_2 = \frac{\partial L}{\partial z_2} = (a_2 - y) \sigma'(z_2)$
- ② **은닉층 오차:** $\delta_1 = (W_2 \delta_2) \odot \sigma'(z_1)$ ($\odot$: 원소별 곱)
- ③ **그래디언트:** $\frac{\partial L}{\partial W_2} = \delta_2 a_1^\top, \quad \frac{\partial L}{\partial W_1} = \delta_1 x^\top$

왜 출력층 오차가 “ $a_2 - y \cdot \sigma'$ ”로 깔끔한가

MSE $L = \frac{1}{2}(a_2 - y)^2$ 일 때:

$$\delta_2 = (a_2 - y) \sigma'(z_2) = (a_2 - y) a_2(1 - a_2)$$

시그모이드 미분항이 **그대로** 곱된다.

교차 엔트로피 $L = -[y \ln a_2 + (1 - y) \ln(1 - a_2)]$ 일 때:

$$\frac{\partial L}{\partial a_2} = -\frac{y}{a_2} + \frac{1 - y}{1 - a_2}$$

$\sigma'(z_2) = a_2(1 - a_2)$ 와 곱하면 **정확히 약분**.

“교차 엔트로피 + 시그모이드 출력”이 표준인 계산적 이유

$\delta_2 = a_2 - y$ — 포화 항(최대 0.25)이 붙지 않아, 예측이 0이나 1에 포화돼도 그래디언트가 상대적으로 크게 유지.

은닉층에서는 약분이 안 일어나 δ_1 에는 여전히 $\sigma'(z_1)$ 가 곱된다 — 9.2절 소실 문제의 시작점.

“역전파”라는 이름의 의미: 오차 벡터의 재사용

- 오차 δ 는 **출력층**에서 계산되고, 은닉층 방향으로 **거꾸로 전파**되면서 각 층의 그래디언트를 만들

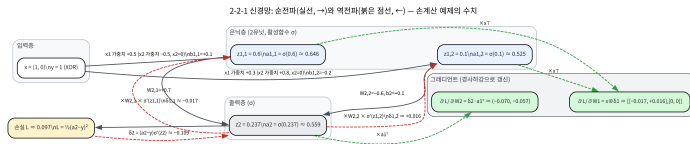
$$\delta_{1,i} = \delta_2 \cdot W_{2,i} \cdot \sigma'(z_{1,i}) \quad \Rightarrow \quad \delta_1 = (W_2 \delta_2) \odot \sigma'(z_1)$$

그래디언트 재사용 — 역전파는 동적 프로그래밍

각 층의 δ 를 **한 번** 계산해 두고 아래 층에 **다시 쓰는** — “같은 미분을 두 번 계산하지 않는” 알고리즘.

- 매 파라미터를 ϵ 씩 흔드는 수치 미분: $O(W)$ 전파 $\times O(W)$ 파라미터 = $O(W^2)$
- 역전파: **순전파 1회 + 역전파 1회** $\approx O(W)$

순전파와 역전파의 흐름: 한 장의 그림



- 실선 = 순전파:
 $z_1 \rightarrow a_1 \rightarrow z_2 \rightarrow a_2$ (입력 → 은닉 → 출력)
- 점선 = 역전파: $\delta_2 \rightarrow \delta_1$ (출력 → 은닉)

역전파는 이름뿐이 아니라, 오차 값이 실제로 층을 거슬러 흘러가는 것. 각 층의 그라디언트 $\partial L / \partial W_2, \partial L / \partial W_1$ 이 여기서 만들어진다.

손으로 한 번: 2-2-1 순전파 + 손실

$$x = (1, 0), y = 1 \text{ (XOR(1,0)=1)}, W_1 = \begin{bmatrix} 0.5 & 0.3 \\ -0.5 & 0.8 \end{bmatrix}, b_1 = (0.1, -0.2), W_2 = (0.7, -0.6), b_2 = 0.1:$$

- **순전파:** $z_1 = x^\top W_1 + b_1 = (0.6, 0.1), a_1 = \sigma(z_1) \approx (0.646, 0.525)$
- $z_2 \approx 0.646 \times 0.7 + 0.525 \times (-0.6) + 0.1 \approx 0.237, a_2 \approx 0.559$
- **손실:** $L = \frac{1}{2}(a_2 - y)^2 \approx 0.097$

역전파:

$$\delta_2 \approx (-0.441) \times 0.559 \times 0.441 \approx -0.109$$

$$\delta_1 = W_2 \delta_2 \odot \sigma'(z_1) \approx (-0.076, 0.065) \odot (0.229, 0.249) \approx (-0.017, 0.016)$$

행=입력 컨벤션: $\partial L / \partial W_1 = x \otimes \delta_1, x_2 = 0$ 이므로 두 번째 행은 0.

2-2-1 손계산: 그래디언트와 한 스텝 갱신

$$\frac{\partial L}{\partial W_2} \approx \delta_2 a_1 \approx (-0.070, -0.057), \quad \frac{\partial L}{\partial W_1} \approx \begin{bmatrix} -0.017 & 0.016 \\ 0 & 0 \end{bmatrix}$$

- 학습률 $\alpha = 0.5$ 로 **한 스텝** 갱신 $\Rightarrow$ 손실 $0.097 \rightarrow 0.087$
- **핵심**: 은닉층 가중치 W_1 까지 **정확히 한 번의 역전파**로 갱신했다

왜 이 계산이 중요하나

PyTorch/TensorFlow는 `.backward()` 한 줄로 이 모든 미분을 자동 계산(자동 미분, 1986 이후의 표준). 하지만 내부에서 연쇄법칙이 정확히 무엇을 하는지 모르면, 9.2절의 소실 문제를 **원인 진단**할 수 없다.

수치 미분 검증: “역전파가 맞다”를 기계로

수치 미분(numerical gradient checking): 가중치를 아주 작게 ($\epsilon = 10^{-6}$) 흔들어서 손실 변화량으로 미분을 직접 재봄

$$\frac{\partial L}{\partial w} \approx \frac{L(w + \epsilon) - L(w - \epsilon)}{2\epsilon}$$

파라미터	역전파	수치 미분	차이
$\partial L / \partial b_2$	-0.1087	-0.1087	6.6×10^{-12}
$\partial L / \partial W_{2,1}$	-0.0702	-0.0702	1.5×10^{-11}
$\partial L / \partial b_{1,1}$	-0.0174	-0.0174	6.3×10^{-12}
$\partial L / \partial W_{1,11}$	-0.0174	-0.0174	6.3×10^{-12}

모든 차이 10^{-11} 수준 = 반중심 차분 오차 한계 $O(\epsilon^2) + O(10^{-16})$ 와 정확히 일치. 새 구조를 직접 구현할 때 실무에서 흔히 쓰는 디버깅 기법. **주의:** W_1 의 행/열 컨벤션이 코드와 다르면 그래디언트가 **전치**되어 “맞는 값인데 왜 안 맞지?” — 역전파가 틀린 게 아니라 컨벤션이 다른 것.

실습: numpy로 2층 신경망 + XOR

```
import math, random

def sigmoid(x):
    return 1 / (1 + math.exp(-x))

def sigmoid_prime(x):
    s = sigmoid(x)
    return s * (1 - s)

def two_layer_forward(x, W1, b1, W2, b2):
    z1 = [sum(W1[i][j] * x[j] for j in range(len(x))) + b1[i]
          for i in range(len(b1))]
    a1 = [sigmoid(v) for v in z1]
    z2 = sum(W2[i] * a1[i] for i in range(len(a1))) + b2
    return sigmoid(z2), (x, z1, a1, z2, sigmoid(z2))

def two_layer_backward(y_true, cache, W2):
    x, z1, a1, z2, a2 = cache
    delta2 = (a2 - y_true) * sigmoid_prime(z2)
    delta1 = [W2[i] * delta2 * sigmoid_prime(z1[i]) for i in range(len(z1))]
    grad_W2 = [delta2 * a1[i] for i in range(len(a1))]
    grad_W1 = [[delta1[i] * x[j] for j in range(len(x))]
                for i in range(len(delta1))]
    return grad_W1, delta1, grad_W2, delta2
```

같은 데이터, 다른 운명: 로지스틱회귀 vs 2층 신경망

같은 XOR 4개, 같은 경사하강법($\alpha = 1.0$, 5000 에폭):

모델	최종 예측	손실
로지스틱회귀 (은닉층 없음)	[0.500, 0.500, 0.500, 0.500]	0.125
2층 신경망 (은닉 4개)	[0.016, 0.977, 0.976, 0.030]	0.0003

- 로지스틱회귀: 네 점을 **전부 0.5(완전 불확신)**으로 예측한 채 0.125에서 **멈춤** — 앞에서 본 부등식 모순이 수렴을 원천 차단
- 2층 신경망: 0.0003까지 내려 네 점을 정확히 분리

구조의 차이가 운명을 정한다

“은닉층이 왜 필요한가”를 가장 극적으로 보여주는 최소 예시.

“XOR을 풀었다”는 말이 성립하려면: 초기화 + 학습률

XOR 학습 성공 = (1) 무작위 초기화가 적절 + (2) 학습률이 적절.

0-초기화 (대칭성 문제): 모든 은닉 유닛이 같은 입력
· 같은 업데이트를 받아 **영원히 같은 값** 유지

0-초기화: 예측 [0.5, 0.5, 0.5, 0.5], 손실 0.125
(멈춤)

출력층이 “같은 값 4개를 합산” = 사실상 1차원. XOR을
분리할 표현력이 **구조적으로 없음**. (9.2절 Xavier/He 해결)

학습률의 “적당함”은 문제별로 다르다. 4점 XOR에서 $\alpha = 0.5$ 가 동작하는 것은 아니다.

학습률 (같은 구조, 무작위 초기화, 시드 고정):

$\alpha = 0.01$: 손실 0.124 (사실상 무학습)

$\alpha = 0.5$: 손실 0.0003 (성공)

$\alpha = 2.0$: 손실 0.0001 (성공, 약간 빠름)

0.01은 50,000 에폭이 돼야 겨우 수렴 — “느리다”가
아니라 “못 배운다”. 2.0은 이 예제에선 수렴하나, 데이터가
크면 발산.

$\alpha = 0.5$ 가 동작한다고 10만 데이터의 CNN에서

자주 하는 실수 (9.1)

- ① **가중치를 0으로 초기화** — 모든 은닉 유닛 출력이 동일해 대칭이 안 깨지고, XOR 포함 어떤 선형비분리 문제도 불가 (실험의 $[0.5, 0.5, 0.5, 0.5]$). 무작위 초기화는 잡음이 아니라 **대칭을 깨는 필수 절차**.
- ② **“작게 하면 안전”** — α 가 너무 작으면 사실상 수렴 불가, 너무 크면 발산(Ch. 2.1). 새로운 구조에서는 $\{0.01, 0.1, 1.0, 10\}$ 후보를 먼저 비교 (9.3의 스케줄링은 이 “적정값”을 학습 진행에 따라 조절).
- ③ **행/열 컨벤션 혼동** — W_1 이 “행=입력, 열=은닉”인지에 따라 그래디언트가 **전치**. “역전파가 틀렸다”고 하기 전에, 수치 미분 코드와 역전파 코드가 **동일 컨벤션**인지 확인.
- ④ **역전파 = “새로운 수학” 오해** — 연쇄법칙의 재사용 (동적 프로그래밍)일 뿐, 새 미분 정리가 아니다.

자주 묻는 질문 (9.1)

- **Q. 은닉층이 하나로 충분한가? 보편근사정리**(hornik, 1991): 은닉층 **하나**(뉴런 수 충분히 많으면)로도 이론적으로 어떤 연속함수든 근사 가능. 하지만 “가능하다” $\neq$ “학습이 쉽다” — 실전은 더 깊은 쪽이 같은 표현력을 **훨씬 적은 뉴런 수**로 달성 (CNN·트랜스포머의 “깊이” 효율성).
- **Q. 역전파가 매번 같은 미분을 계산한다는 확신은? 수치 미분**으로 $-\frac{L(w + \epsilon) - L(w - \epsilon)}{2\epsilon}$ 가 역전파 값과 소수점 몇 자리 내면 일치.
- **Q. 연쇄법칙인데 왜 1986년에야?** 아이디어는 1974년 워브루브스키 박사논문까지 거슬림 — 1986 논문은 **대중화**: (a) 최우도 추정과 연결, (b) 문자 인식 실험, (c) “은닉층을 학습할 수 있다”의 메시지.
- **Q. 순전파 vs 역전파 계산량? 동급** — 둘 다 $O(\text{총가중치})$. 한 에폭 = 순전파 1회 + 역전파 1회 $\approx 2 \times$ 순전파.

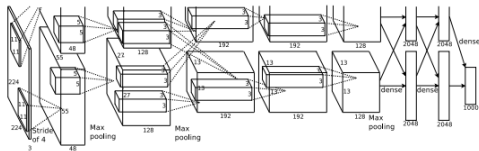
확인 문제 (9.1)

- ① **XOR의 부등식 모순.** $w_2 + b > 0, w_1 + b > 0, b \leq 0, w_1 + w_2 + b \leq 0$ — 첫 두 식을 더해 셋째 식과 모순을 만들어, 이 네 식을 모두 만족하는 (w_1, w_2, b) 가 **존재하지 않음**을 보여라.
- ② **δ_1 의 재사용.** $\delta_1 = (W_2 \delta_2) \odot \sigma'(z_1)$ 에서 δ_2 가 출력층에서 이미 계산돼 **재사용** 된다는 것이 역전파가 “동적 프로그래밍”인 이유. 10층 망에서 모든 δ_l 을 처음부터 다시 계산하면 몇 번의 미분이 필요한가, 재사용하면 몇 번인가?
- ③ **교차 엔트로피의 약분.** 시그모이드 + 교차 엔트로피면 $\delta_2 = a_2 - y$ 로 미분항이 약분. MSE 때의 δ_2 와 비교하고, 그 차이가 “포화된 예측에서도 그래디언트가 살아있는” 것과 어떻게 연결되는지 한 문장으로.

9.2 그래디언트 소실·폭발, 활성화함수, 초기화

시그모이드의 숨은 함정: 그래디언트 소실

- 역전파는 이론적으로 층이 몇 개든 그래디언트를 계산 가능
- 실제로 10개, 20개로 깊게 쌓으면: **입력 가까운 층은 학습이 거의 진행되지 않음** — 깊을수록 오히려 못 배운다
- 1990년대: 르쿤의 MNIST 네트워크도 **2~3층**에 불과, “딥러닝”이라는 말 자체가 2006년 힌턴까지 사실상 사라짐



AlexNet (2012, ImageNet 승리): 5 합성곱 + 3 완전연결 = 8층 — 그 “깊음” 자체가 뉴스였다.

원인: $\sigma'(z) = \sigma(z)(1 - \sigma(z))$ 의 **최댓값은 $z = 0$ 에서 0.25**.
역전파는 층을 거슬러 이 미분값을 곱해서 전달.

$$\underbrace{0.25 \times 0.25 \times \dots}_{10 \text{ 개 층}} \approx 9.5 \times 10^{-7}$$

→ **그래디언트 소실(vanishing)**: 신호가 사실상 0.
반면 가중치 초기화가 잘못되면 층을 거칠수록 오히려 **지수적으로 커지는 그래디언트 폭발(exploding)**도 일어남.

왜 “곱”인가 — 할인율 비유

- L 이 층 1의 W_1 에 영향을 미치는 경로: $W_1 \rightarrow z_1 \rightarrow a_1 \rightarrow z_2 \rightarrow \cdots \rightarrow L$ — 연쇄법칙이 이 사슬을 따라 미분을 **연속으로 곱**
- 곱 연산은 **할인율**처럼: 각 층이 신호의 일부만 전달하면 10층에 0.25^{10} 으로 지수 감소
- 비유: 10명이 순서대로 “들린 것의 25%만” 다음 사람에게 속삭이면 — 10번째에게는 100만분의 1도 안 남음

핵심

소실은 “느리다”가 아니라 **“지수적으로 0에 수렴한다”**.

소실과 폭발은 **같은 연쇄법칙 곱의 두 얼굴**: 각 층의 “승수”가 1보다 작으면 소실, 크면 폭발. 이 승수는 활성화함수(σ')와 가중치(W_l)의 크기로 — 이번 절의 “활성함수”와 “초기화·정규화”가 바로 이 승수를 제어하는 것.

L개 층의 곱 형태 + 손으로 체감

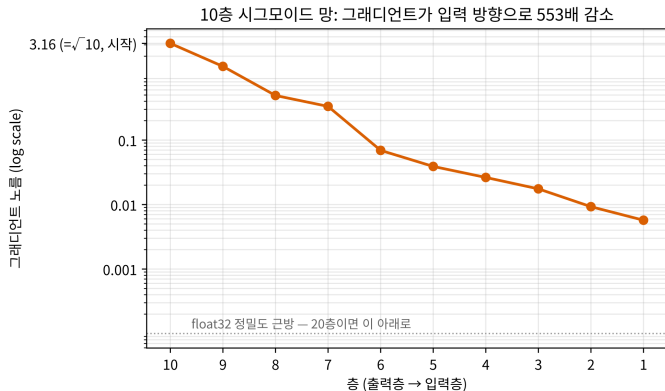
$$\frac{\partial L}{\partial W_1} \propto \prod_{l=2}^L \sigma'(z_l) \cdot W_l$$

- 각 $\sigma'(z_l) < 1$ (시그모이드) $\Rightarrow$ 지수적으로 0 $\rightarrow$ **소실** | 각 항이 > 1 (가중치 큼) $\Rightarrow$ 지수 증가 $\rightarrow$ **폭발**

상황	값	의미
$\sigma' = 0.2$ 3개 층	$0.2^3 = 0.008$	그라디언트의 0.8%만 1층에 도달
$\sigma' = 0.2$ 10개 층	$\approx 1.0 \times 10^{-7}$	사실상 0
$\sigma' = 0.2$ 20개 층	$\approx 1.05 \times 10^{-14}$	float32 이내 $\rightarrow$ 반올림 0
승수 평균 2, 20개 층	$2^{20} \approx 10^6$	백만 배 부풀림, 발산 (NaN 전조)

$\sigma'(z) = 0.2$ 는 $z \approx \pm 1.1$ — 시그모이드 곡선이 “비포화”인 전형 지점. 20층만 거치면 가중치가 발산하고 학습 중 NaN이 된다.

실습: 10층 시그모이드에서 실제로 사라지는 그래디언트



- 꼭 10 시그모이드 층 10개, $W \sim \mathcal{N}(0, 1)$
- 출력층 3.16 → 입력층 0.0057, **약 550배 감소**
- $0.25^{10} \approx 9.5 \times 10^{-7}$ 은 활성 미분만 곱한 극단 하한 — 가중치 곱($\|W\| \approx \sqrt{10}$)이 일부 보상해 550배로 관측
- 실제 평균 $\sigma'(z)$ 는 0.13~0.19 — z 가 $\pm 2 \sim 6$ 까지 벌어진 **포화** 뉴런이 많기 때문

시그모이드 미분이 “0.25 이하”인 것 자체보다, **실제 데이터 위에서 포화로 평균이 0.2를 훨씬 하회하는** 것이 더 치명적 — 입력 근처 층은 사실상 그래디언트를 못 받아 학습이 멈춘 것과 다름없음. 20층이면 float32 정밀도($\approx 10^{-8}$) 이내로 **그래디언트 자체가 0으로 반올림된다**.

활성함수가 왜 필요한가

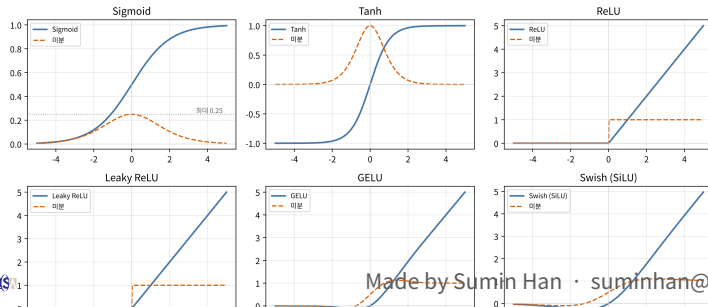
- 활성함수가 없으면: $a_2 = W_2(W_1x) = (W_2W_1)x$ — 층을 아무리 쌓아도 한 번의 선형 변환과 같음
- 9.1에서 배운 것처럼 선형 모델로는 XOR이 **원칙적으로 불가** — 활성함수 없이는 100층을 쌓아도 영원히 못 풀음
- 활성함수는 각 층 사이에 “곡선을 접어 넣는” **비선형성**을 제공, 층이 쌓일수록 표현 가능 공간이 **지수적으로** 넓어짐

1958년 퍼셉트론의 스텝함수($z \geq 0$ 이면 1, 아니면 0)가 역사적으로 가장 첫 번째 “활성함수” — 이 맥락에서 보면 자연스러움. 본 절의 활성함수·초기화와 9.3의 정규화는 전부 “깊은 망에서도 그래디언트가 살아서 전달되게 하려면”이라는 하나의 문제의 서로 다른 각도의 답이다.

기본 3종: Sigmoid / Tanh / ReLU

함수	정의	미분	특징
Sigmoid	$\frac{1}{1 + e^{-z}}$	최댓값 0.25 ($z = 0$)	출력 (0, 1), 확률 해석 유용하나 소실에 취약
Tanh	$\frac{e^z - e^{-z}}{e^z + e^{-z}}$	최댓값 1	출력 $(-1, 1)$, sigmoid보다 낮지만 양 끝에서 포화
ReLU	$\max(0, z)$	$z > 0: 1, z < 0: 0$	양수 구간 미분 항상 1 — 층을 거쳐도 작아지지 않음

활성함수(실선)와 미분(점선) — 미분이 1 근처를 유지하는가?



여섯 가지 활성화함수(윗줄 sigmoid/tanh/ReLU, 아랫줄 Leaky ReLU/GELU/Swish)와 각 미분 — sigmoid 미분의 최댓값 0.25(점선)과 ReLU 미분의 0/1 양극화가 대비.

Sigmoid와 Tanh의 한계

Sigmoid — 두 가지 문제

- 1 **포화:** $|z| \gtrsim 3$ 이면 $\sigma'(z) < 0.05$ — 깊은 망에서 소실을 가속
- 2 **중심화 문제:** 출력이 항상 $(0, 1)$ 의 양의 값 $\Rightarrow$ 다음 층의 가중치가 **동일 방향**으로 움직이는 경향 $\Rightarrow$ 비대칭 골짜기, 학습 느려짐

Ch.2의 로지스틱회귀에서는 확률 해석 때문에 시그모이드가 **옳지만**, 은닉층에서는 확률 해석이 불필요한데도 두 불이익만 받음.

Tanh — 시그모이드를 -1 방향으로 중심화

$$\tanh(z) = 2\sigma(2z) - 1$$

- 출력 $(-1, 1) \Rightarrow$ 중심화 문제 **해소**
- 미분 최댓값 1 = 시그모이드(0.25)의 **4배**
- 1990년대 은닉층의 **사실상 표준**
- 그러나 $|z| \gtrsim 3$ 에서 여전히 포화 ($0.985 \rightarrow$ 미분 ≈ 0.03) — **깊이**에는 부족

ReLU: 2012년 이후 은닉층의 사실상 표준

- $f(z) = \max(0, z)$ — 미분이 $z > 0$ 에서 **정확히 1**이라 곱해도 줄어들지 않음
- **계산이 가장 단순**: 지수함수 없음, 비교 연산 하나
- 음수 구간 출력이 정확히 0 $\Rightarrow$ **희소(sparsity)** — 활성화된 뉴런만 신호 전달, 계산 빨라짐
- 2012년 크리스베스키 일행: “rectifier는 학습 시간을 **크게 줄인다**” — 같은 해 **AlexNet**이 ReLU를 써 ImageNet을 휩쓸며 은닉층 **사실상 표준**

자주 하는 실수: 로지스틱회귀의 습관으로 은닉층에 시그모이드를

확률 해석이 필요한 것은 **출력층뿐**. 은닉층의 기본값은 **ReLU 계열**, 출력층은 **문제 형태에 맞게**:
회귀 $\rightarrow$ 항등, 이진분류 $\rightarrow$ 시그모이드, 다중분류 $\rightarrow$ softmax.

Dying ReLU — ReLU의 아킬레스건

- $z < 0$ 일 때 미분이 **정확히 0** — 뉴런이 “죽으면” **영원히** 부활 불가 (시그모이드 포화는 미분이 작지만 0은 아님 — 성격이 다름)
- 정상 망에서는 $z \sim \mathcal{N}(0, 1)$ 의 절반이 음수라 약 50%가 “숨어” — 이 자체는 **설계값**. “dying”은 이 비율이 **비정상적으로 치솟는 것**.

실험 (실습 노트 검증, $y = \sin x_1 + \frac{1}{2}x_2^2$ 회귀, 음수 쪽 치우친 입력, 은닉 40 ReLU, SGD, 학습률만 변경):

ReLU 학습률=0.45: 죽은 뉴런 52% $\rightarrow$ 97% (100에폭) loss 0.26 고착

Leaky ReLU 학습률=0.45: 죽은 뉴런 0% (정의상 불가) loss 0.095 계속 감소

“미분이 0”은 시그모이드 포화처럼 “느리게만” 만드는 게 아니라 **학습 경로의 한쪽을 완전히 차단**. 큰 학습률/BatchNorm 부재/outlier 입력 상황에서 죽은 뉴런이 돌이킬 수 없이 누적. (Adam 스케일 조정 + BatchNorm이 z 흔들림을 잡아주므로 실무에선 흔치 않지만, “죽으면 되살릴 수 없다”는 **구조적** 취약함은 변하지 않음.)

ReLU 계열 변형 (2013년경부터)

죽는 뉴런을 “완전 사망”이 아닌 “숨을 쉰다”로 만든 것:

함수	정의	한 줄 설명
Leaky ReLU	$\max(0.01z, z)$	음수 미분 0.01 — 죽지 않고 “숨 쉰다”
ELU	$\min(0, \alpha(e^z - 1))$	음수 구간 $-\alpha$ 로 완만 — 미분 0이 아님
Softplus	$\log(1 + e^z)$	ReLU의 매끄러운 근사 — $\text{softplus}' = \sigma$
Swish (SiLU)	$z \cdot \sigma(z)$	$z < 0$ 에서 작은 음수 허용 — ReLU보다 부드럽고, 깊은 망에서 손실 더 낮음 (2017)
GELU	$z \cdot \Phi(z)$	“ z 가 얼마나 양수적인가”의 확률로 가중 — 트랜스포머(GPT/BERT)의 표준 은닉층 활성화함수 (Ch. 12)

GELU 기억해 두면: Ch. 12에서 “왜 GELU인가”는 바로 이번 절의 논리(미분이 1 근처 유지 + 부드러움 → 최적화 안정) 위에 얹혀 있다.

가중치 초기화: 0이 왜 치명적인가

- $w_{ij} = 0$ 모두이면 $a_l = f(b_l)$ 로 층마다 상수 출력, 역전파 시 각 뉴런 그래디언트가 동일 $\Rightarrow$ 모든 가중치가 동일하게 갱신 $\Rightarrow$ 대칭이 영원히 안 깨짐
- 결과: 폭 100짜리 층 하나가 폭 1짜리 뉴런 하나와 완전 같은 표현력 (9.1의 0-초기화 XOR 실험과 동일)

기준: 활성값의 2차 모멘트 $\mathbb{E}[a^2]$ 를 층을 거치며 보존

$z_l = \sum_i w_{li} a_{l-1,i}$, w 가 $\mathbb{E}[w] = 0$, 분산 σ_w^2 이면:

$$\mathbb{E}[z_l^2] = n_{l-1} \sigma_w^2 \mathbb{E}[a_{l-1}^2] \quad (n_{l-1} = \text{fan-in})$$

이를 1로 $\Rightarrow \sigma_w = 1/\sqrt{n_{l-1}} = \text{Xavier (Glorot, 2010) 초기화}$.

$\mathbb{E}[a^2]$ 를 쓰는 이유: 정규화(9.3)가 조절하는 대상이 “스케일” = 2차 모멘트. $\tanh$ 같은 0-중심·대칭 활성화함수에서는 $\mathbb{E}[a^2] = \text{Var}(a)$ 라 “분산 보존”과 동일.

He 초기화: ReLU가 버리는 ½을 2배로

- ReLU는 입력의 절반($z < 0$)을 0으로 버림 $\Rightarrow$ 2차 모멘트가 **정확히 절반**: $\mathbb{E}[\text{ReLU}(z)^2] = \frac{1}{2} \mathbb{E}[z^2]$
- 이 ½을 보상 $\Rightarrow$ 가중치 분산 **2배**: $\sigma_w = \sqrt{2/n_{l-1}}$ = He (Kaiming He 등, 2015, ResNet 논문) 초기화

측정 (실습 노트, fan-in 100, 한 층 통과 후 $\mathbb{E}[a^2]$ 승수):

조합	승수	해석
ReLU + Xavier	0.499	매 층 정확히 ½ $\rightarrow$ 10층 후 $0.5^{10} \approx 10^{-3}$ (소실)
ReLU + He	0.998	$\frac{1}{2} \times 2 = 1$, 보존 (안정)
tanh + Xavier	0.393	선형 근사(1)보다 작음 — 포화로 추가 압축

“2”라는 숫자가 정확히 ReLU가 버리는 ½을 보상한다는 것을 **정확한 수치**로 확인. tanh+Xavier가 0.39인 것은 포화 압축 — 실무에서 tanh도 Xavier를 쓰는 건 1:1 보존이 1차 근사이고, 미세한 스케일 드리프트는 9.3 BatchNorm이 흡수.

초기화 요약 + “표준편차를 감으로”의 대가

초기화	표준편차	잘 맞는 활성화함수	핵심 아이디어
Xavier (2010)	$1/\sqrt{\text{fan-in}}$	tanh, sigmoid, linear	$\mathbb{E}[a^2]$ 를 1:1 보존 (선형 가정)
He (2015)	$\sqrt{2/\text{fan-in}}$	ReLU 계열	ReLU가 버리는 $1/2$ 을 2배로 보상

실험: 표준편차로만 다른 10개 ReLU 층(폭 100) 통과 후 활성화값 분산:

$\sigma = 1.0$: $1.002 \rightarrow 34 \rightarrow 1.6 \times 10^3 \rightarrow \dots \rightarrow 1.1 \times 10^{17}$ (폭발)

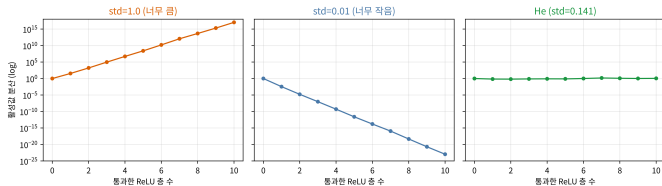
$\sigma = 0.01$: $1.002 \rightarrow 3.4 \times 10^{-3} \rightarrow \dots \rightarrow 1.1 \times 10^{-23}$ (소실)

He ($\sigma \approx 0.141$): $1.002 \rightarrow 0.688 \rightarrow 0.650 \rightarrow 0.759 \rightarrow \dots \rightarrow 1.114$ (안정)

분포는 Gaussian이나 Uniform($k = \sqrt{3}\sigma$) 모두 가능. PyTorch nn.Linear 기본값: ReLU 계열 $\rightarrow$ kaiming_uniform_, 그 외 $\rightarrow$ xavier_uniform_ — **프레임워크가 이미 이번 절의 결론을 기본값으로**. “적당해 보이는 숫자를 감으로 고르는 것”과의 차이가 이 정도다.

세 가지 초기화의 “운명”이 그림으로

초기화 표준편차 → 활성값 분산의 운명 (폭 100 ReLU 10층, seed=0)



- $\sigma = 1.0$: 층마다 약 30배씩 부풀어 10^{17} (**폭발** → 그래디언트 폭발)
- $\sigma = 0.01$: 약 300배씩 줄어 10^{-23} (**소실** → 학습 멈춤)
- **He**: 0.65~1.5로 안정

주목할 세부: He도 정확히 1이 아니라 0.65~1.5 사이를 **방향** — “½ 보상”이 정규분포 가정 하의 근사이므로 완전하지 않다. 이 “마지막 여정”의 미세 스케일 드리프트를 학습 중에도 붙잡아 주는 것이 9.3 BatchNorm.

큰 그림: 그래디언트를 살리는 세 라인 방어선

“깊은 망을 학습 가능하게 만드는 것”은 세 가지가 함께 작동할 때 성립:

- ① **활성함수**: 미분이 1 근처를 유지하는 것을 고른다 (ReLU 계열) → “할인율” 자체를 낮춤
- ② **초기화**: 각 층의 입출력 스케일(분산)을 보존 (Xavier/He) → 시작점부터 “할인율 $\times$ 레버”가 1 근처로 맞춤
- ③ **정규화 (9.3)**: 학습 도중 스케일이 다시 흐르지 않게 붙잡아 줌 (BatchNorm, Dropout) — **그래디언트 클리핑**은 노름이 상한을 넘으면 잘라내는 하드 안전장치, **Adam 등 현대 최적화기법**은 스케일 불균형을 부분 흡수

“더 깊은 망”이 “더 똑똑한 모델”이 되려면, 연쇄법칙의 곱이 각 층에서 **1 근처**로 유지되어야 한다. 활성함수(미분의 크기) · 초기화(스케일의 시작점) · 정규화(학습 중 스케일 유지) — 이번 절과 9.3의 모든 기법은 이 하나의 조건을 서로 다른 각도에서 지키는 도구다.

실험: “활성함수 + 초기화” 실패의 두 얼굴

같은 구조(은닉 7층 × 폭 16, 출력 1), 4점 XOR, 1500에폭, SGD 학습률 0.5, MSE — 활성함수만 변경(초기화는 각각에 맞는 값):

구성	결과
tanh (Xavier)	ep 0: loss=0.3796 → ep 50: loss=NaN (폭발 — 발산)
ReLU (He)	ep 0: loss=0.4453 → ep 1500: loss=0.2500 (정체 — 0.5 상수 예측)

- tanh: 그래디언트 **폭발** → NaN — “무언가가 터졌다”는 **명확한** 신호
- ReLU: 0.2500 = “라벨 평균 $\bar{y} = 0.5$ 를 상수로 예측”의 MSE — **가중치가 하나도 학습되지 않은 상태와 정확히 같은 손실**
- “loss가 0.25로 수렴했다”고 보고하면 성공처럼 보임 — 전형적 **정체(stagnation)** 실패

참고: 7층 XOR은 평범한 SGD로 **안장점(saddle point)**에 갇히기 쉬우며, Adam(lr=0.01)을 추가하면 같은 구조가 실제로 수렴한다 (실습 노트). “활성함수+초기화”만으로 완전 해결되지 않음.

확인 문제 (9.2)

- ① **계산.** 20층 시그모이드, 각 층 $\sigma'(z_l) \leq 0.25$ 이면 그래디언트는 $(1/4)^{20} = 2^{-40} \approx 9.1 \times 10^{-13}$ — 1조분의 1의 1%도 안 됨. 왜 “실제” 감소는 이 값과 정확히 같지 않은가? (힌트: 곱에는 σ' 만이 아니라 W_l 도 — $\|W_l\| > 1$ 이면 보상, < 1 이면 추가 감소)
- ② **코딩.** `he_init_std =  $\sqrt{2/\text{fan-in}}$ , xavier_init_std =  $1/\sqrt{\text{fan-in}}$` 완성. 검증: `e2_out(1, 100, he, relu)` ≈ 1.0 (보존), `e2_out(1, 100, xavier, relu)` ≈ 0.5 (소실), `e2_out(1, 100, he, tanh)` ≈ 2.0 (선형 근사 매 층 2배).
- ③ **개념.** 폭 $100 \rightarrow 100 \rightarrow 100$ ReLU에 Xavier(0.1)로 초기화, 10층 반복 시 $\mathbb{E}[a^2]$ 는 $0.5^{10} \approx 0.001$ — 소실인가 폭발인가? 그리고 He가 “2”를 왜 곱하는지 ReLU의 어떤 성질에서 오는지 한 문장으로.

자주 묻는 질문 (9.2)

- **Q. ReLU 미분이 0/1인데 왜 “완전히” 안 되나?** 곱의 인자가 σ' 하나뿐이 아님: (1) W_l 크기가 나쁘면 ReLU여도 소실/폭발 — 초기화의 역할, (2) $z < 0$ 이면 미분 0이라 죽은 뉴런은 영구히 기여 0 — ReLU 변형이 보완, (3) 학습 도중 스케일이 다시 흐름 — BatchNorm의 역할. **“활성함수+초기화+정규화” 세 라인이 함께 작동해야 해결.**
- **Q. 프레임워크 기본값이면 초기화 신경 안 써도 되나?** 전형적 구조(완전연결 ReLU)에는 맞다 — PyTorch 기본값이 Xavier/He. 하지만 (a) 스킵 커넥션/레지듀얼, (b) 커스텀 층 (합성곱, LSTM 셀)에서는 “fan-in”이 명확히 정의되지 않아 기본값이 적용 안 됨. “학습이 안 된다/발산한다”를 디버깅할 때 **첫 질문은 항상 “각 층의 그래디언트·활성 노름을 재보라”.**
- **Q. 출력층 활성화함수는?** 문제 형태가 결정: 회귀 → 항등(MSE 짝), 이진분류 → 시그모이드(교차엔트로피 짝), 다중분류 → softmax(Ch. 17 언어모델에서 재회).
- **실습 노트북:** 10층 시그모이드 $553 \times$ 감소, 세 초기화의 분산 갈림, dying ReLU 97%, 활성화함수 6종 비교, 7층 XOR tanh+Xavier(NaN) vs ReLU+He(0.25) vs Adam(수렴).

9.3 정규화 기법과 학습률 스케줄링

과적합과 신경망 전용 정규화

- **과적합**: 학습 데이터는 완벽히 맞히는데 새 데이터에서 성능 저하 — Ch. 4.1 편향-분산 관점에서, 파라미터가 매우 많은 신경망은 **극단적으로 유연(분산 큼)**해 특히 취약
- Ch. 6의 L1/L2 정규화는 그대로 쓸 수 있으나, 신경망은 **구조 자체를 이용한** 정규화도 함께 사용:

Dropout

- 학습 시 매 스텝마다 일부 뉴런을 확률 p 로 무작위로 꺼버림
- 뉴런 간 **공적응(co-adapt)**을 억제 — 특정 뉴런에만 의존하는 대신 **견고한(robust)** 특징 학습

“정규화(regularization)”라는 말은 릿지 회귀 (Tikhonov, 1963)에서 왔고, 2010년대 dropout·BN 등장 이후 “손실을 바꾸는 L1/L2” + “**계산 구조 자체를 바꾸는 구조적 정규화**”라는 새 축이 생김.

Batch Normalization (BN)

- 각 층의 입력을 **미니배치** 단위로 평균 0, 분산 1로 정규화, 이후 학습 가능한 γ, β 를 다시 곱·추가
- **internal covariate shift** 감소 → 학습이 **더 빠르고 안정적**

Dropout 코드: masking + 스케일 보정

```
def dropout(activations, p, training):  
    if not training:  
        return activations  
    import random  
    return [0.0 if random.random() < p else a / (1 - p)  
            for a in activations]
```

왜 $/(1 - p)$ 로 스케일을 보정하는가

학습 때 평균적으로 뉴런의 $(1 - p)$ 만 살아있으므로, 추론 시 전체 뉴런을 다 쓰는 것과 기댓값을 맞추기 위함 (inverted dropout).

Dropout: Srivastava-Hinton 등 2014, 아이디어는 Hinton 2012 (co-adaptation 방지).

Dropout: 앙상블 관점 — 2^N 개의 서브네트워크

- 한 전파에서 살아남는 뉴런의 **조합**이 매번 달라 $\Rightarrow N$ 개 이진 뉴런의 층에서 이론상 2^N 개의 “서브네트워크” 존재
- $N = 10$ 이면 1024개, 은닉층 3개면 **수백만~수십억** 개 — dropout은 이 모든 서브네트워크를 **파라미터를 공유한 채** 동시에 학습
- 앙상블 = 구성원이 서로 다른 실수를 해서 평균냈을 때 편차 감소(Ch. 4.1) — dropout은 “수억 개를 만들되 파라미터를 공유”해서 그 이득을 **지수적으로 싸게** 얻는 장치

왜 이것이 “과적합 억제”인가

뉴런이 서로의 존재를 전제로 학습(co-adapt)하지 못하게 됨 — “나만 잘하면 돼”가 아니라 “**나를 꺼놔도 팀이 돌아가야 하니까**” $\Rightarrow$ 특정 뉴런에 집중해 잡음까지 외우던 (과적합) 대신, 어떤 뉴런이 빠져도 **견고하게 동작하는 분산된 특징** 학습.

1/(1-p) 보정이 정확히 보존하는 것

활성값 $a = (1, -2, 3, 4)$, $p = 0.5$ (4개 평균을 내려침으로 전달한다고 가정):

- **보정 없이**: $E[\text{출력}] = (1 - p) a = 0.5 a$ — 평균 0.75로 절반. 추론 때는 뉴런 전부 커 a (평균 1.5)를 사용 $\Rightarrow$ **학습(0.75)과 추론(1.5)이 $(1 - p)$ 배 어긋남**
- **$/(1 - p)$ 로 보정**: $E[\text{mask} \cdot a / (1 - p)] = E[\text{mask}] a / (1 - p) = (1 - p) a / (1 - p) = a$ — **학습 때 기댓값이 정확히 추론값과 같음**

실제로 모든 $2^4 = 16$ 개 마스크 조합의 평균 = **정확히 1.5**, 무작위 20만 번 샘플링 = 1.5012 (스크래치패드 검증). “1.5 = 1.5” — 이 **불변(invariance)**이 inverted dropout의 정석. **추론 때는 masking을 아예 끄고, 모든 뉴런을 켜되 각 출력을 $(1 - p)$ 로 스케일** — 학습 때의 2^N 양상블 평균을 한 번의 전파로 근사.

p 선택: 은닉층 $p = 0.2 \sim 0.5$ 가 흔함, 과적합이 심할수록 p 키움. $p \geq 0.7$ 이면 살아남는 뉴런 부족 $\rightarrow$ **과소적합** — “train·val 정확도가 함께 떨어지기 시작하는 지점”에서 멈춤. 입력층에 dropout을 쓰는 경우는 드뭄.

Dropout: 그림 + 자주 하는 실수

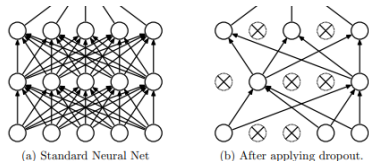


Figure 1: Dropout Neural Net Model. **Left:** A standard neural net with 2 hidden layers. **Right:** An example of a thinned net produced by applying dropout to the network on the left. Crossed units have been dropped.

posterior probability given the training data. This can sometimes be approximated quite well for simple or small models (Xiong et al., 2011; Salakhutdinov and Mnih, 2008), but we would like to approach the performance of the Bayesian gold standard using considerably less computation. We propose to do this by approximating an equally weighted geometric mean of the predictions of an exponential number of learned models that share parameters

(a) 표준 신경망 vs (b) 드롭아웃 적용 “얇아진”
신경망 — 훈련 중 랜덤 제거되는 유닛 (원 논문
Figure 1).

자주 하는 실수: 추론 때도 `model.train()`을
켜둔 채

PyTorch는 `.train()` 모드에서만 masking을 켜고,
`.eval()`에서 끄고 $(1 - p)$ 스케일로 동작.
`model.eval()`을 잊으면 **매번 다른 예측 + 값이**
 $(1 - p)$ 배 축소된 “학습용” 분포 → 정답률에
미치고도 정확도가 이상하게 뚝. “테스트마다
결과가 달라지는” 정석 원인.

검증/추론 루프는 항상 `with torch.no_grad(): model.eval();`
...로 감싼다.

Batch Normalization: 각 층 입력을 매 스텝 재조정

BatchNorm (Ioffe-Szegedy, 2015): 각 층의 **입력**을 미니배치 단위로 평균 0, 분산 1로 정규화한 뒤, **학습 가능한 스케일·시프트**를 다시 붙임:

$$\hat{x} = \frac{x - \mu_B}{\sqrt{\sigma_B^2 + \epsilon}}, \quad \text{출력} = \gamma \hat{x} + \beta$$

- μ_B, σ_B^2 : 그 미니배치의 평균·분산, γ (스케일)· β (시프트): 학습되는 파라미터
- γ, β 가 있는 이유: 정규화가 표현력을 해칠 수 있으니, 필요 시 “정규화를 일부 취소”할 자유도를 남겨주는 보험

원 논문 Figure 1: BN이 MNIST 테스트 정확도 상승을 크게 가속화하고, 더 큰 학습률을 쓸 수 있게 함.

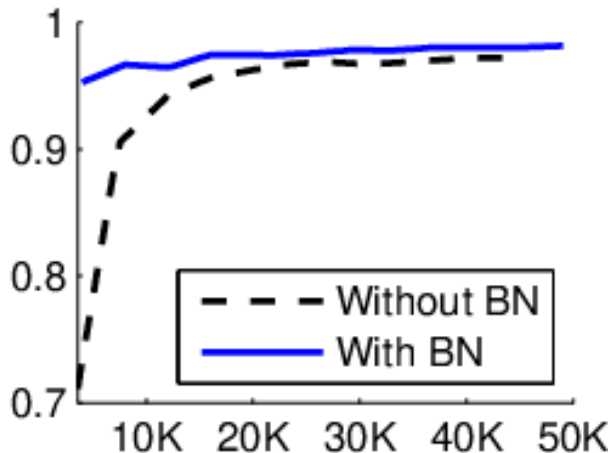
BN이 학습을 빨리 하는 이유 — 그리고 한 발 더

- 이전 층 가중치가 매 스텝 갱신될수록 뒤층이 보는 입력 분포가 계속 바뀜(mean/분산 미끄러짐 = **internal covariate shift**) — 뒤층이 매번 “파라미터를 다시 맞춰야” 하는 이중 작업, 큰 학습률 불가, 초기화에 예민
- BN은 각 층 입력을 매 스텝 (0, 1)로 재설정: (1) 큰 학습률을 안전하게, (2) 초기화에 훨씬 덜 민감, (3) batch 통계에 노이즈가 담겨 **가벼운 정규화 효과**까지
- 9.2 연결: BN이 활성화값을 $\mathcal{O}(1)$ 로 묶어 **그래디언트 소실·폭발이 훨씬 어려워짐** — “깊은 망을 실제로 학습시킬 수 있게” 하는 또 다른 각도의 답

한 발 더 정직하게

원 논문은 속도의 주된 원인을 “internal covariate shift 감소”로 꼽았으나, 후속 분석(Santurkar 등, 2018)은 손실함수의 **조건 개선(landscape 매끄러워짐)**이 진짜 주요 요인이라고 보정. 둘 다 사실 — “분포 안정화”는 가장 쉬운 직관.

BN 그림 + 이중 모드(학습 vs 추론)



BatchNorm 유무에 따른 MNIST 학습 수렴 속도 (원 논문 Figure 1) — BN이 더 큰 학습률로 적은 스텝에 고 정확도 도달.

꼭 알아야 할 세부: 이중 모드

학습 때는 **현재 미니배치**의 μ_B, σ_B^2 사용.

추론 때는

- 1 배치가 없을 수 있음 (batch size = 1)
- 2 한 배치 통계는 그 배치만의 “요행/사고” 포함

⇒ 학습 중 **러닝 평균·분산**(지수 이동 평균)을 계속 기록해 두고, 추론 때는 그 누적된 통계를 사용.

“학습/추론 시 통계가 다르다”는 사실은 dropout의 `.train()/.eval()`과 **같은 범주의 함정** — BN을 직접 구현할 때 가장 많이 틀리는 부분.

BN의 한계: 작은 배치 — LayerNorm/InstanceNorm/GroupNorm

- BN은 **batch 통계**에 의존 — batch size가 1~2 로 작으면 통계가 극도로 불안정, BN이 제대로 동작 안 함
- **LayerNorm**: 배치가 아니라 **특징(feature) 축**을 정규화 — 시계열·NLP(트랜스포머, 작은 배치/변동 길이) 표준
- **InstanceNorm**: batch를 무시, 샘플·채널 단위 (생성 모델)
- **GroupNorm**: 채널을 그룹 단위로 묶어 정규화 — batch size에 의존하지 않음

“BN을 쓰면 무조건 좋은가?”

배치가 충분히 큰 분류/회귀 CNN에서는 대체로 Yes, 그렇지 않은 곳에서는 LayerNorm 계열.

자주 하는 실수: batch 1~2 (시계열 RNN, 배치 1 추론)에서 BN을 고집 — batch 통계가 1~2개 샘플로 추정돼 분산이 0 근처 폭주/무의미. 이 경우 **LayerNorm이 정석**.

가중치 감쇠: Ch. 6의 L2가 최적화기로 바뀐 얼굴

- Ch. 6의 $L2(\lambda \|w\|^2)$ 는 손실함수 자체를 바꿈. 신경망에서는 같은 일을 **최적화기 옵션**으로 하는 **weight decay**가 표준 — PyTorch `SGD(weight_decay=...)`, AdamW 등

- 직관:

$$w \leftarrow (1 - \eta wd) w$$

매 스텝 **모든 가중치를 $(1 - \eta wd)$ 배로 살짝 줄임** — “가중치를 끊임없이 0 방향으로 살짝 당겨, 불필요하게 큰 가중치를 허용하지 않는” 규칙

- 예: $wd = 0.001, \eta = 1 \Rightarrow$ 스텝마다 0.999배 $\Rightarrow$ 1000스텝 뒤 $0.999^{1000} \approx 0.368 \approx 1/e$ (실행 검증)

= Ch. 6의 λ 가 **최적화 규칙으로 재포장**된 것.

L2 페널티 vs “해결된(decoupled)” weight decay

L2 페널티

$$w \leftarrow w - \eta (\nabla L + \lambda w)$$

손실에 $\lambda \|w\|^2$ 를 붙여 그래디언트에 λw 를 포함.

- ∇L 이 w 의 선형 함수(이차 손실)이면 두 갱신이 숫자적으로 일치 (이차 손실, $\eta = 0.1$, $wd = 0.01$ 한 스텝: 두 방식 모두 정확히 **0.99880000** — 실행 검증)
- 비선형 손실, 특히 Adam처럼 adaptive 최적화기에서는 L2 페널티의 “ λw 를 adaptive 스케일로 다시 나누는” 동작이 의도치 않게 효과 왜곡 $\Rightarrow$ Adam에서는 AdamW(분리된 weight decay)가 표준 (Loshchilov–Hutter, 2017)

“L2 = weight decay 아닌가?”의 정직한 답: “이차 손실에서는 같고, Adam에서는 다름 — 그래서 AdamW가 있다.” weight decay는 **가중치의 크기**를 벌 (손실함수 쪽), dropout은 **뉴런의 공적응**을, BN은 **층간 분포**를 건들 (계산 구조 쪽) — 서로 다른 축이므로 **함께** 쓰인다.

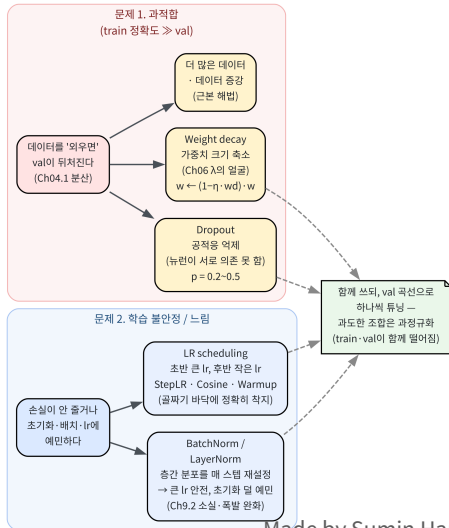
해결된 weight decay (AdamW 방식)

$$w \leftarrow w - \eta \nabla L \quad \text{이후} \quad w \leftarrow (1 - \eta wd) w$$

손실 그래디언트만 내린 뒤 **곱셈**으로 줄임.

도구 지도: 어떤 문제에 어떤 도구

신경망 정규화·학습률 스케줄링 도구 지도 — '과적합'과 '학습 불안정' 두 문제별, 어떤 도구가 어떤 축을 푸는가



- 실무 두 축: (a) **과적합** (train 높고 val/test 뒤흔침), (b) **학습 불안정** (손실 안 줄거나 초기화·배치·lr에 예민)
- **과적합 문제면**: dropout + weight decay + 근본 해법인 **더 많은 데이터·증강**
- **느리다/불안정하면**: Batch/LayerNorm(분포 안정 → 큰 lr 가능) + **학습률 스케줄링**
- **“모두 쓰면 될까요?”**: 아니 — BN과 dropout은 둘 다 약한 정규화 효과를 내, 과용 시 **과정규화(train·val 함께 떨어짐)**. 실무: **하나 추가 → val 곡선 확인 → 다음 추가.**

학습률 스케줄링: 큰 lr으로 시작, 작은 lr으로 끝

- Ch. 2.1: α 가 너무 크면 **발산**, 너무 작으면 **느림** — 신경망에서는 이 둘을 **동시에** 얻는 법
- **학습률 스케줄링**: 초반 큰 α 로 빠르게 손실 줄임 $\rightarrow$ 진행할수록 α 를 줄여 골짜기 바닥 근처에서 **정교하게 수렴** — “학습률이 크면 최솟값 근처에서 튕겨 나간다”를 **후반부에서만** 피하는 절충안
- 흔한 방식: **StepLR** (에폭마다 일정 비율로 감소), **코사인 스케줄** (부드러운 감소, Loshchilov–Hutter 2016 SGDR)

왜 — 골짜기에서 볼링공: 최솟값 근처의 손실은 **좁고 가파른 골짜기**. 큰 스텝 $\rightarrow$ 바닥을 **건너뛰고** 반대편 벽에 부딪혀 **진동**(바닥에 못 착지). 작은 스텝은 천천히지만 **정확하게** 착지. 반면 최솟값에서 **멀리** 있을 때는 작은 스텝만으론 너무 느림. $\Rightarrow$ **멀리 있을 때는 크게(빠르게), 가까이 왔을 때는 작게(정교하게)**.

골짜기에서 직접 확인 + 왜 “한 번에 큰 lr”이 위험한가

1차원 포물선 $L(w) = (w - 3)^2 + 0.5$, 시작 $w = 0$, 300스텝:

전략	결과
작은 lr 고정 (0.0072)	300스텝 뒤 $ w - 3 \approx 0.039$ — 바닥에 못 도착 (예산 소진)
0.08 (200스텝) $\rightarrow$ 0.0072 (100스텝)	$ w - 3 \approx 2 \times 10^{-15}$ — 도착

진동 직접 보기: 같은 포물선에 $\alpha = 0.9$ (최적 0.5 초과)로 1스텝 반복:

$$w : 0.0 \rightarrow 5.4 \rightarrow 1.08 \rightarrow 4.54 \rightarrow 1.77 \rightarrow 3.98 \rightarrow 2.21 \rightarrow 3.63 \rightarrow 2.50 \rightarrow \dots$$

최솟값 3을 오차로 왔다갔다하며 점진적으로만 수렴 — 큰 lr은 “빠르다”지만 **바닥에 못 붙는다**.

이 포물선은 볼록해 큰 lr도 결국 수렴하지만, 실제 비볼록 손실에서는 **한 번에 너무 큰 lr**은 최솟값을 넘나들며 발산하거나 나쁜 극소점에 갇힌다.

StepLR / 코사인 / warmup — 모양을 기억하자

고정 lr(회색 수평선) / StepLR(파랑 계단) / 코사인(빨강,
300에폭 동안 0.01 $\rightarrow$ 0으로 부드럽게).

- **StepLR**: step_size 에폭마다 $\alpha \leftarrow \gamma \alpha$. 예:
 $\alpha_0 = 0.01, \gamma = 0.5, \text{step}=50 \Rightarrow$
 $0.01 \rightarrow 0.005 \rightarrow 0.0025 \rightarrow 0.00125$
- **코사인**: $\alpha_t =$
 $\alpha_{\min} + \frac{1}{2}(\alpha_{\max} - \alpha_{\min})(1 + \cos(\pi t/T_{\max}))$
— 계단 뚝뚝보다 “lr의 공백기” 없음, 후반
수렴 매끄러움
- **warmup**: 처음 몇 에폭은 **아주 작은 lr**으로
시작해 점차 늘린 뒤 decay

왜 warmup이 필요한가 — 무작위로 막 초기화된 가중치에 Adam 같은 **adaptive 최적화기**를 처음부터 큰 lr으로 돌리면, 첫 스텝들의 그래디언트/2차 통계가 아직 무의미한데 큰 업데이트가 나가 **초반 손실이 오히려 발사**. 짧은 warmup이 이 첫 스텝들을 안정 — **GPT/트랜스포머 학습에서 warmup이 표준인** 이유가 바로 이것.

초기 학습률 α_0 고르는 실무 기준

기준: “가장 큰, 하지만 손실이 안정적으로 감소하는 lr”. $\{10^{-4}, 10^{-3}, 10^{-2}, 10^{-1}, 10^0\}$ 스윙 → 손실이 꾸준히 줄어드는 가장 큰 lr 선택 → 스케줄링으로 후반에 감소.

moons 데이터(300개, 64-64-64 MLP, SGD, 150에폭), lr만 스윙:

α	0.001	0.01	0.1	0.2	0.5
train acc	0.770	0.837	0.860	0.857	0.860
val acc	0.771	0.832	0.852	0.850	0.846

- $\alpha = 0.001$: 150에폭이 지나도 0.77 — 배우지 못함(과소적합)
- 0.1 ~ 0.2에서 최고(0.85); $\alpha = 0.5$ 는 오히려 나빠짐(0.846) — “큰 것이 나을 것”이라는 직관이 여기 깨짐
- “안정적으로 감소하는 가장 큰 lr” = 0.1~0.2, 스케줄링으로 후반에 줄이면 최적

자주 하는 실수: (1) 스케줄링 없이 처음부터 너무 큰 lr → 발산, (2) 너무 작은 lr → “이 구조는 안 배우는 것 아닌가” 착각 (실제로 lr 문제), (3) Adam으로 **warmup 없이** 큰 lr → 초반 손실 발사. 셋 다 “학습이 안 된다/나빠진다”의 동일 증상 — 원인을 lr 쪽에서 먼저 확인.

실습: PyTorch로 정규화·스케줄링 적용 MLP

```
import torch, torch.nn as nn

class RegMLP(nn.Module):
    def __init__(self):
        super().__init__()
        self.net = nn.Sequential(
            nn.Linear(2, 64), nn.BatchNorm1d(64), nn.ReLU(),
            nn.Dropout(0.3),
            nn.Linear(64, 64), nn.BatchNorm1d(64), nn.ReLU(),
            nn.Dropout(0.3),
            nn.Linear(64, 64), nn.BatchNorm1d(64), nn.ReLU(),
            nn.Dropout(0.3),
            nn.Linear(64, 1))
    def forward(self, x):
        return self.net(x).squeeze(-1)

opt = torch.optim.SGD(model.parameters(), lr=0.1)
sched = torch.optim.lr_scheduler.StepLR(opt, step_size=100, gamma=0.3)
# 학습루프안에서 : opt.step() 다음에 sched.step() 호출
```

make_moons(선형분리 불가능) 데이터, 64-64-64 은닉층, 300 에폭. 순서: Linear → BatchNorm → ReLU → Dropout
— dropout을 BN보다 먼저 하면 미니배치마다 꺼지는 뉴런이 달라 BN의 평균·분산 계산 자체가 불안정해짐.

결과: “train이 함께 오른 정규화”

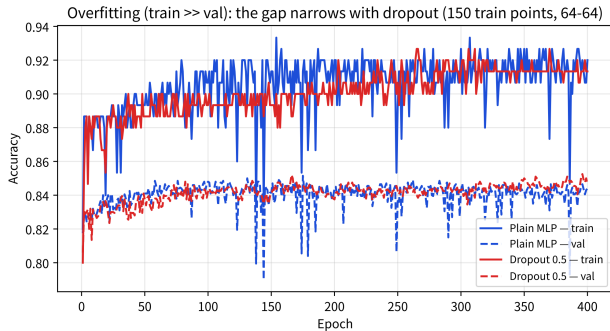
300에폭 학습 후 (미니배치 순서에 따라 숫자는 시드에 소폭 변동, 패턴 고정):

모델	train acc	val acc
일반 MLP (정규화·스케줄링 없음)	0.847	0.846
정규화 MLP (dropout + BN + StepLR)	0.870	0.846

- 정규화 모델이 **train 정확도까지 더 높음** — “정규화 = 모델을 단순하게 = train 하락”이라는 오개념을 바로잡는 예
- train이 올랐다는 것: dropout의 “단순화”가 아니라 **BatchNorm이 학습 자체를 더 안정적으로 만들어서 더 잘 (많이) 배운** 결과
- val이 “떨어지지 않았다”는 것: 이 조합은 **과정규화에 빠지지 않는 안전한 지점**

(실습 노트 § 4) 에폭별 train/val 곡선으로 “일반 모델은 학습 초반에 멈추고, 정규화 모델이 꾸준히 train을 올리다가 val을 지킨다”는 것을 눈으로 확인.

dropout이 gap을 좁힌다 (숫자로)



학습 데이터만 150개(작게)로 제한한 moons, 64-64 MLP, 400에폭:

	train	val
평범한 MLP	0.920	0.844
dropout 0.5	0.913	0.846

gap: **0.076** → **0.067** — train·val 간격이 좁혀지고 val이 **오름**. “gap이 벌어지는 것 = 과적합”이라는 Ch. 4.1 신호를 dropout이 직접 좁혀줌.

BatchNorm이 활성값의 흔들림을 없앤다 (숫자로)

“internal covariate shift”가 실제로 일어나는지 — 입력 스케일 1→3으로 변하도록 만들고 **2층에 들어가는 활성값의 표준편차**를 추적(실행 검증, 실습 노트 § 3):

구성	ep 0	ep 20	ep 40	ep 60	ep 80
BN 없음	0.437	0.557	0.693	0.920	1.084
BN 있음	0.595	0.576	0.580	0.577	0.591

- BN 없음: 2층이 보는 활성값이 **0.44** → **1.08로 2.5배** 미끄러짐 = 뒤층이 매번 다시 맞춰야 하는 internal covariate shift
- BN 있음: **0.58 부근으로 안정**

“층의 입력 분포를 매 스텝 재설정한다”가 숫자로

이 표가 바로 9.2의 “BN이 활성값을 $\mathcal{O}(1)$ 로 묶어 그래디언트 소실·폭발이 어려워진다”를 실증하는 숫자.

확인 문제 + 자주 묻는 질문 (9.3)

- **확인 1 (1/(1-p)의 불변성)**: $a = (1, -2, 3, 4)$, $p = 0.5$ — “보정이 없으면 학습 기댓값이 $(1 - p)$ 배 줄어 추론값과 어긋난다”를 수식으로 보이고, 보정 후 $E[\text{학습}] = \text{추론}$ 임을 확인.
- **확인 2 (BN 이중 모드)**: 추론 시 BN이 러닝 평균/분산을 쓰고 현재 배치 통계를 쓰지 않는 이유(배치 부재, 배치 통계 노이즈) — dropout의 `.train()/.eval()`과 같은 범주의 함정임을 한 문장으로.
- **확인 3 (L2 vs weight decay)**: 어떤 손실에서는 같고, Adam에서는 왜 다르고, 그래서 AdamW가 등장하는지.
- **확인 4 (스케줄링의 기하)**: 포물선 $(w - 3)^2$ 의 overshoot(진동)으로 “왜 큰 lr로 시작해 작은 lr으로 끝내는가” — “최솟값 근처 큰 스텝 = 바닥 건너뛰기”.

FAQ: Dropout+BN 순서는 Linear $\rightarrow$ BN $\rightarrow$ ReLU $\rightarrow$ Dropout 권장. λ (Ch.6)는 “얼마나 단순해야 하는가”(손실함수)를, 스케줄은 “얼마나 빠르게/정교하게 찾아갈 것인가”(최적화 속도)를 조절 — 서로 다른 축. BN은 약한 정규화 효과가 있어 과적합이 심한(데이터 적은) 문제에서는 dropout도 함께. LayerNorm은 batch 축을 특징 축으로 — batch 작거나 시계열/순서 표준. weight decay + dropout은 흔히 함께, 단 한꺼번에 강하게 쓰면 과소적합 — val 곡선으로 하나씩 강도 올리기. 스케줄링 없이는 “초반 큰 lr의 속도 + 후반 작은 lr의 정교함”을 한 값으로 잡을 수 없음.

이번 주차 정리

- ① **9.1 — 은닉층이 XOR의 벽을 넘는다:** 부등식 모순으로 단층 퍼셉트론의 한계를 보이고, 은닉층이 “직선으로 갈리는 새 공간”을 만든다. 역전파 = 연쇄법칙의 **재사용** (동적 프로그래밍) — 수치 미분으로 구현을 검증한다.
- ② **9.2 — 그래디언트는 “할인율의 곱”:** 층마다 $\sigma' \times W$ 를 곱해 깊어지면 지수적으로 0(소실)이거나 불어나 (폭발). 세 라인 방어: **활성함수(ReLU 계열) + 초기화 (Xavier/He) + 정규화(BatchNorm)**.
- ③ **9.3 — 학습을 실제로 끝내는 도구:** dropout(공적응 억제) · BatchNorm(분포 안정, 큰 lr 가능) · 가중치 감쇠(AdamW) · 학습률 스케줄링(초반 크게, 후반 작게, warmup).

다음 주 — Chapter 10. 합성곱 신경망 (CNN)

이번 장의 “일반부”(역전파, 활성함수, 초기화, 정규화) 위에, **이미지의 특성에 맞는 “특수부”** — 합성곱층 — 를 올린다. 이번 장에서 세운 역전파·활성함수·초기화·정규화·스케줄링은 그 뒤의 모든 장(CNN→RNN→트랜스포머→LLM)에서 다시 쓰이는 도구다.